CS544 - Topics In Networks

Understanding and Profiling CXL.mem using

PathFinder

SIGCOMM 2025 · Xiao Li et al. UW-Madison & Intel

Presented By:

230101012 - Annigandla Kameshwara Rao
230101067 - Muduguru Mahedar Reddy

OVERVIEW

Presentation Outline

01 **Problem Statement**

Why CXL memory profiling matters

02 **Motivation & Gaps**

What existing tools cannot do

03 **System Model**

Server as a Clos network

04 **PathFinder Techniques**

4 core mechanisms explained

05 **7 Case Studies**

Real-world experiments & findings

06 **Strengths & Weaknesses**

Critical analysis

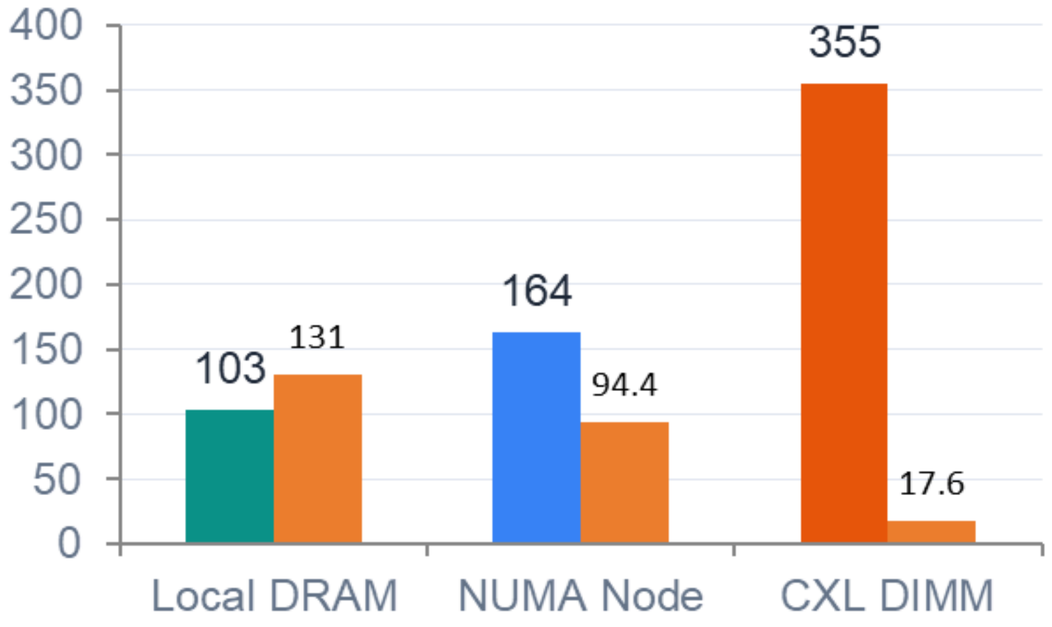
What is CXL.mem and Why Does It Matter?

CXL (Compute Express Link) is a high-speed interconnect built on PCIe that lets CPUs directly access remote DIMMs via load/store instructions as if the memory were local.

The problem:

- CXL memory is 3-4× slower than local DRAM (355 ns vs 103 ns latency).
- Queue-based hardware (SB, LFB, CHA) gets congested - blocking even local memory.
- This changes the application's data locality and working set, especially when multiple CXL and local memory flows co-exist.
- No tool exists to profile CXL.mem end-to-end across the full hardware path.

Measured on Intel SPR server (Intel MLC):



Memory access latency comparison (ns), Bandwidth (GB/s)

Why Existing Tools Are Not Enough

No existing tool provides end-to-end, path-level diagnostics across the CPU pipeline, cache hierarchy, and CXL fabric simultaneously.

Intel VTune / AMD uProf

What it does: Micro-architecture profilers using Top-Down Analysis

Gap: On-chip only, cannot link core-level slowdowns to off-chip CXL memory access

Memory benchmarks (STREAM, MLC)

What it does: Measure latency-bandwidth curves under different access patterns

Gap: Report performance numbers but cannot diagnose WHY no path-level breakdown

PCIe / NoC profilers

What it does: Characterize interconnect traffic and PCIe transactions

Gap: Capture in-flight data but cannot tie movements back to source core or destination cache

CHALLENGES

Why is it difficult to build such utility?

01

CXL.mem has several non-transparent data paths, originating from core or caches, traversing a chain of on-chip hardware modules non-deterministically (depending on data locality), and finally hitting an off-chip memory device, whose execution characteristics hinge on compute, memory, and I/O substrates.

02

CXL.mem load/store instructions are tightly coupled with the processor execution, which has multiple deep and out-of-order pipelines, runs at the nanosecond-scale granularity, and only exposes a few programmable interfaces to query the execution telemetry information online

03

Local and CXL memory flows share the underlying hardware, whose data paths are overlapped and intertwined. It is hard to isolate their executions, separate their architectural behaviors and performance impacts, and determine whether CXL memory access is the culprit.

Main Contribution: PathFinder

A systematic, informative, and lightweight end-to-end CXL.mem profiler

Clos Network Model

Models the server + chipset as a multi-stage Clos network - using well-known networking telemetry on hardware

232 PMU Counters

Identifies and maps 232 hardware performance counters across core, CHA, uncore, and CXL device

4 Profiling Techniques

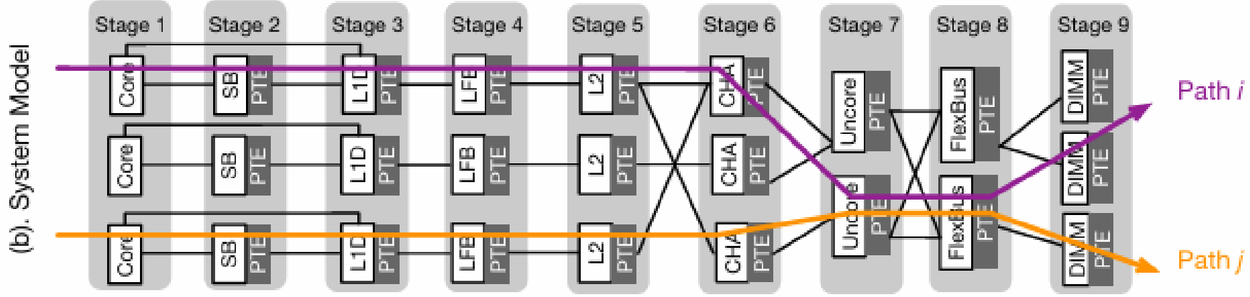
PFBuilder, PFEstimator, PFAnalyzer, PFMaterializer - path construction to multi-snapshot analysis

7 Case Studies

Validated on 77 real-world applications on Intel SPR and EMR servers with Intel Agilex and Micron CZ120 CXL DIMMs

The Server as a Multi-Stage Clos Network

S1	S2	S3	S4	S5	S6	S7	S8
Core	**SB**	**L1D/LFB**	**L2**	**CHA/LLC**	**Uncore**	**FlexBus**	**CXL DIMM**
v_src	store buf	L1 cache	private	v_int	mesh	M2PCIe	v_dst



Memory Flow (mFlow)

Core_i ↔ lDIMM_j / cDIMM_j
All load/store/prefetch on that path, bounded by Core# × DIMM#

App-bound

Lifetime = workload duration

Location-aware

New mFlow if thread migrates core

Bidirectional

Tracks request + response

4 CXL.mem Data Paths (per mFlow):

#1 DRd

Demand read - misses all caches, reaches CXL DIMM

#2 DWr / RFO

Write through store buffer, RFO gains exclusive ownership

#3 RFO load

Read for ownership from L1D/L2/LLC chain

#4 HW/SW PF

Hardware/software prefetch - async DRd or RFO to DIMM

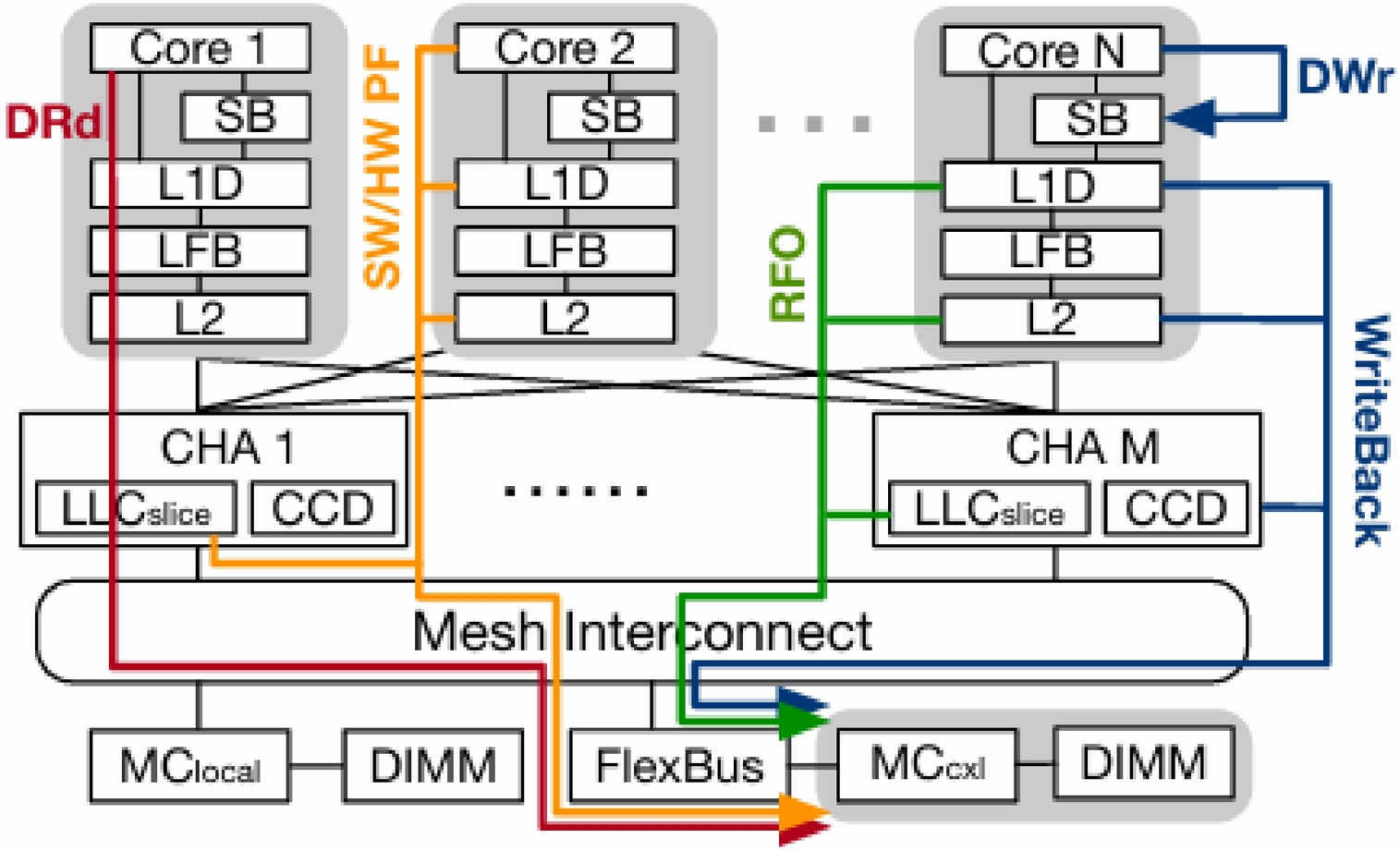


Figure 1: Four `CXL.mem` data paths on an Intel SPR/EMR processor. SB=Store Buffer. LFB=Line Fill Buffer. CCD=Cache-Coherent Directory. CHA=Caching and Home Agent.

Four Core Profiling Techniques

01 PFBuilder *≈ Like Traceroute*

- Constructs the CXL data path map by synthesizing PMU hit/miss counters at each hardware stage.
- Uses the TOR (Table of Requests) in the CHA - equivalent to a forwarding table - to trace which core's request went to which LLC slice and then to which DIMM.

02 PFEstimator *≈ Like Reverse Traceroute*

- Attributes CXL-induced stall cycles bottom-up from CXL DIMM to Core.
- Proportionally distributes stall time based on traffic load at each hop - isolating the CXL contribution from local memory stalls.

03 PFAnalyzer *≈ Little's Law (Queueing)*

- Models each hardware component as an FCFS queue.
- Computes $L = \lambda \times W$ (queue length = arrival rate × service time) to find the bottleneck.
- Identifies culprit paths causing head-of-line blocking at congested components.

04 PFMaterializer *≈ Like Flow Telemetry (NetFlow)*

- Stores time-series snapshots in InfluxDB.
- Runs Flux queries to detect data locality trends, contention patterns, and resource under-utilization across snapshots - like network operators running dashboards over flow records.

01 PFBuilder *≈ Like Traceroute*

Hit Location	DRd			RFO			HW PF			DWr		
	FOTS	GCCS-s1	GCCS-s2	FOTS	GCCS-s1	GCCS-s2	FOTS	GCCS-s1	GCCS-s2	FOTS	GCCS-s1	GCCS-s2
SB										7.8E+08	8.5E+07	2.1E+09
L1D	4.7E+09	8.1E+08	4.5E+09									
LFB	3.1E+08	2.8E+08	1.3E+08									
L2	4.3E+07	1.7E+07	1.7E+08	4.4E+06	3.8E+05	2.1E+07	1.8E+08	1.4E+08	1.7E+08	2.3E+05	84345	1.1E+07
local LLC	5.4E+06	5.0E+06	1.3E+07	7.5E+04	7.7E+04	1.0E+06	2.5E+07	9.7E+06	5.9E+07			
snc LLC	9.4E+05	4.0E+05	8.1E+06	6.1E+03	1.0E+04	8.4E+05	1.8E+07	8.1E+06	6.3E+07	1.5E+08	6.5E+06	7.9E+07
remote LLC	13385	5290	6984	586	42	379	19031	10920	22646			
CXL Memory	2.5E+07	1.9E+06	8.1E+06	1.5E+06	935	4.1E+06	2.2E+08	1.8E+08	2.1E+07			

Table 7: We classify mFlows into DRd, RFO, HW PF and DWr data paths and separate path hit distribution on SB, L1D, LFB, and L2 components for 649.*fotonik3d_s* **and** 602.*gcc_s*, **where GCCS-s1 and GCCS-s2 are two selected snapshots from** 602.*gcc_s*. **For mFlows entering uncore region, we separate hit distribution on local LLC, SNC LLC, remote LLC, and CXL memory. L1D and LFB counters only provide hit count data at load path granularity.**

02 PFEstimator ≈ *Like Reverse Traceroute*

Algorithm 2 The PFEstimator Algorithm.

```
procedure PIPELINE_STALL_BREAKDOWN (CXL_DIMM_ID)
    id = CXL_DIMM_ID; AllLoads = 0;
    CXL_DIMM_Stall = get_CXLDIMM_stall_cycle(id);
    for j in all_FlexBus_RCs do                ▷ CXL DIMM → FlexBus RC
        FlexBusRC_Load[j] = load_agg_rc2dimm(j, i);
        AllLoads += FlexBusRC_Load[j];
    for j in all_FlexBus_RCs do
        weight = FlexBusRC_Load[j] / AllLoads;
        FlexBusRC_Stall[j] = weight × CXL_DIMM_Stall;
    for i in all_FlexBus_RCs do                ▷ FlexBus RC → Host Uncore
        AllLoads = 0;
        FlexBusRC_Stall [i] += get_FlexBusRC_stall_cycle(i);
        for j in all_Host_Uncores do
            HostUncore_Load[j] = load_agg_uncore2rc(j, i);
            AllLoads += HostUncore_Load[j];
        for j in all_Host_Uncores do
            weight = HostUncore_Load[j] / AllLoads;
            HostUncore_Stall[j] = weight × FlexBusRC_Stall[i];
    for i in all_Host_Uncores do               ▷ Host Uncore → CHA
        AllLoads = 0;
        HostUncore_Stall [i] += get_Uncore_stall_cycle(i, id);
        for j in all_CHAs do
            CHA_Load[j] = load_agg_cha2uncore(j, i);
            AllLoads += CHA_Load[j];
        for j in all_Host_Uncores do
            weight = CHA_Load[j] / AllLoads;
            CHA_Stall[j] = HostUncore_Stall[i] × weight;
    CHA→LLC and in-core path segments are omitted;
```

03 PFAnalyzer ≈ *Little's Law (Queueing)*

Algorithm 1 The Delay-based PFAnalyzer Algorithm.

```
HAL: the hardware abstract representation that contains its PMU
counter statistics and all housed path/mFlow information;
Q: queue length array;
procedure HW_OCCUPANCY_DETECTION(HAL module)
    for p in module.all_cxl_paths do
        for c in module.all_components do
            if c == L2 or c == L1D or c == LLC then
                λ_hit = c.hits[p]/c.clocks;
                λ_miss = c.misses[p]/c.clocks;
                W_hit = c.delay_hit;
                if c == LLC then
                    W_miss = c.delay_miss;                    ▷ LLC
                else
                    W_miss = c.delay_tag;                     ▷ core L2,L1D
                Q[p][c]=λ_hit*W_hit+λ_miss*W_miss;
            else                                              ▷ core LFB,RAM
                λ_hit = c.hits[p]/c.clocks;
                W_hit = c.delay_hit;
                Q[p][c]=λ_hit*W_hit;
    culprit_path = MAX_OCC(Q);
```

7 Case Studies

Evaluated on 77 applications · Intel SPR + EMR servers · Agilex & Micron CZ120 CXL DIMMs

Case 1 — PFBuilder
Path Classification

Case 2 — PFEstimator
Pipeline Stall Breakdown

Case 3 — PFAnalyzer
Local vs CXL Interference

Case 4 — PFEstimator + PFAnalyzer
Concurrent CXL Contention

Case 5 — PFAnalyzer + PFBuilder
Bandwidth Partition

Case 6 — PFMaterializer
Data Locality

Case 7 — All 4 Techniques
Performance Optimization

Path Classification & Pipeline Stall Breakdown

Case 1 - Path Classification (PFBuilder)

- PFBuilder reads PMU hit/miss counters hierarchically across all stages.
- Uses TOR entries in CHA to map core → LLC slice → DIMM destination.
- Reports per-path traffic load for DRd, RFO, HWPF, DWr paths.
- Example (649.fotonik3d): HWPF accounts for 59.3% of uncore accesses and CXL hits are 8.1× more than local LLC hits
- Reveals that CXL memory heavily redirects access via hardware prefetch paths

Case 2 - Pipeline Stall Breakdown (PFEstimator)

- Back-propagates CXL-induced stall cycles from DIMM up to core - like reverse traceroute.
- CXL stalls decrease by ~75% from FlexBus toward L1D due to cache locality absorbing the impact
- For FFT: stalls are 42.7% at FlexBus+MC and 40.3% at CXL DIMM - dominated by uncore
- For raytrace: FlexBus stall alone is 67.1% of total - a clear link bottleneck
- SB benefits most from L1 locality - DWr stalls drop 90.6% from FlexBus to SB

Case 2 - Pipeline Stall Breakdown (PFEstimator)

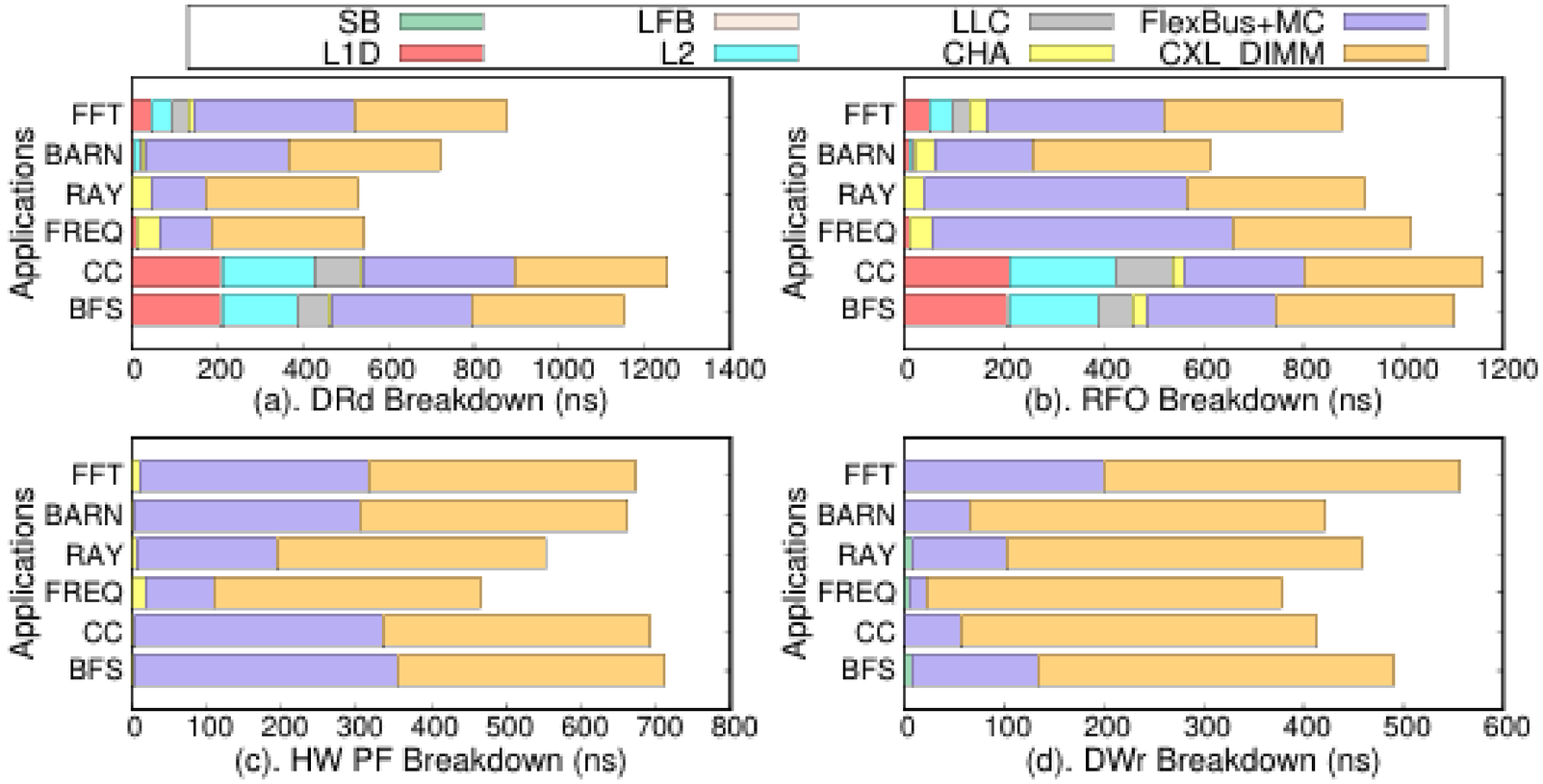


Figure 6: We break down CXL-induced stall cycles of SB, L1D, LFB, L2, CHA/LLC, FlexBus+MC, and CXL DIMM.

Interference Analysis - Local vs CXL & Concurrent CXL

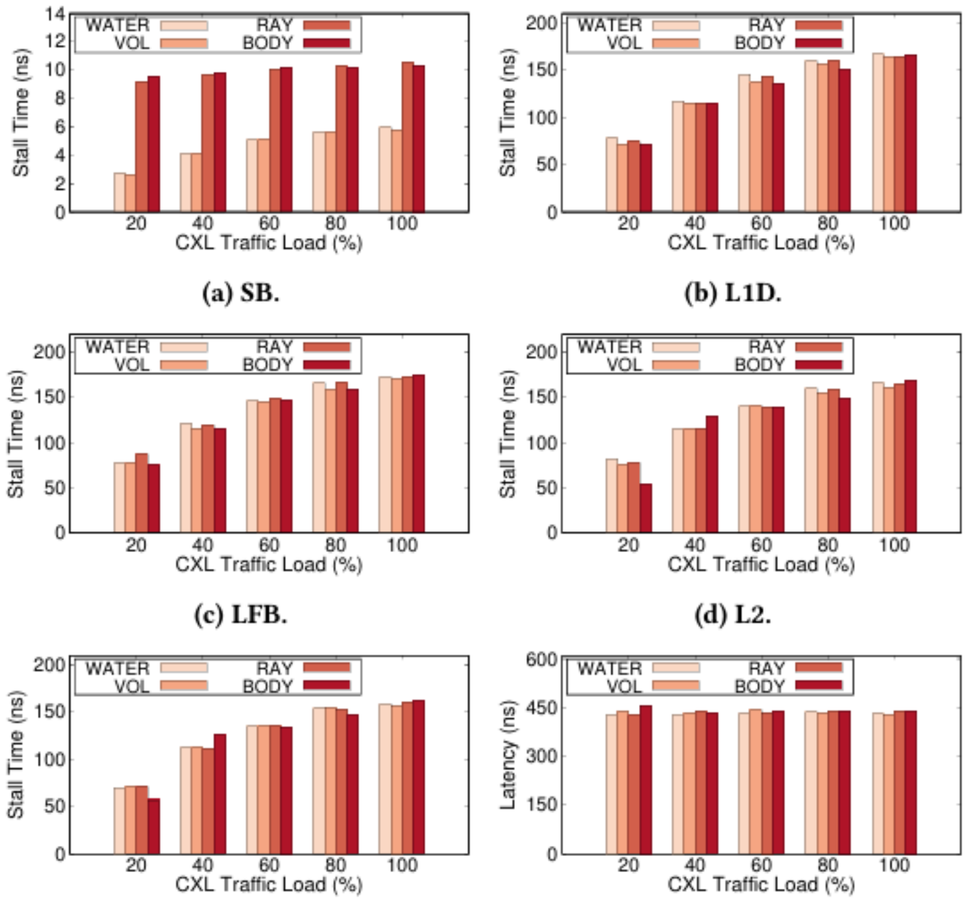
Case 3 - Local vs CXL Interference

- A local mFlow and CXL mFlow placed on the same core - CXL load varied 20% → 100%
- FlexBus and CHA remain uncongested - bottleneck is inside the core.
- CXL-induced stall increases: SB 1.7×, L1D 2.2×, LFB 2.2×, L2 2.4×, LLC 2.4×
- CXL slows pipeline → degrades L1D locality → L2 queue builds up.
- Bottleneck shifts from DRd on L1D to DRd on L2 as CXL load grows.

Case 4 - Concurrent CXL Contention

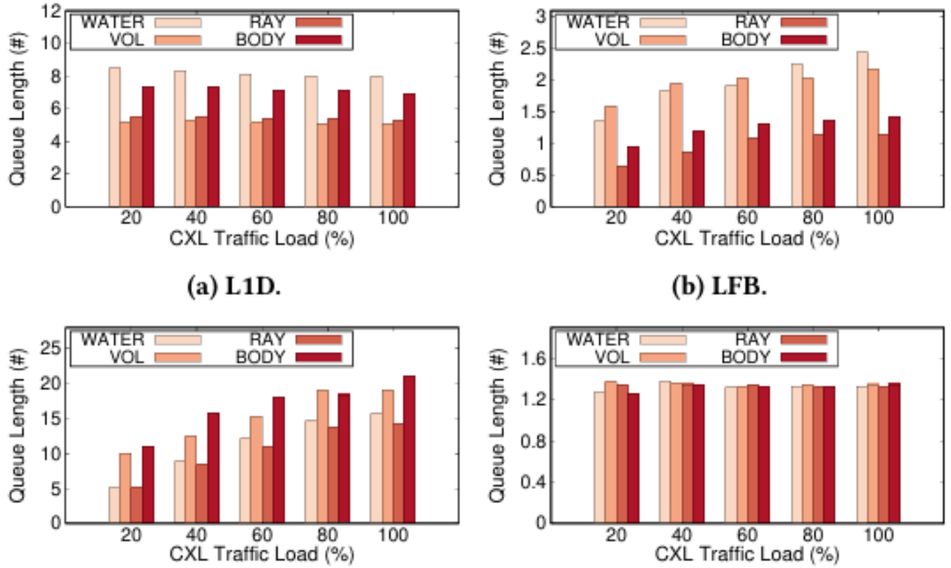
- Multiple cores each running CXL mFlows - YCSB benchmark, load 20% → 100%
- All flows converge at FlexBus+MC: 4.3× latency increase observed there.
- Contention propagates backward: LLC stall +1.8×, SB +2.1×, LFB +2.9×, L2 +1.8×
- YCSB throughput drops by 77.4% on average as CXL traffic increases.
- PFAnalyzer detects bottleneck shift from DRd on L1D to HWPF on FlexBus+MC.

Case 3 - Local vs CXL Interference



(a) SB. (b) L1D. (c) LFB. (d) L2. (e) Core LLC. (f) FlexBus+MC (DRd).

Figure 7: We present the pipeline stall cycle as varying the CXL traffic load. (a), (b), (c), (d), (e), and (f) report the behavior for SB, L1D, LFB, L2, core LLC, and FlexBus+MC, respectively.



(a) L1D. (b) LFB. (c) L2 (DRd). (d) FlexBus+MC (DRd).

Figure 8: We present the component queue length as varying the CXL traffic load. (a), (b), (c), (d) report the behavior for L1D, LFB, L2, and FlexBus+MC, respectively.

Case 4 - Concurrent CXL Contention

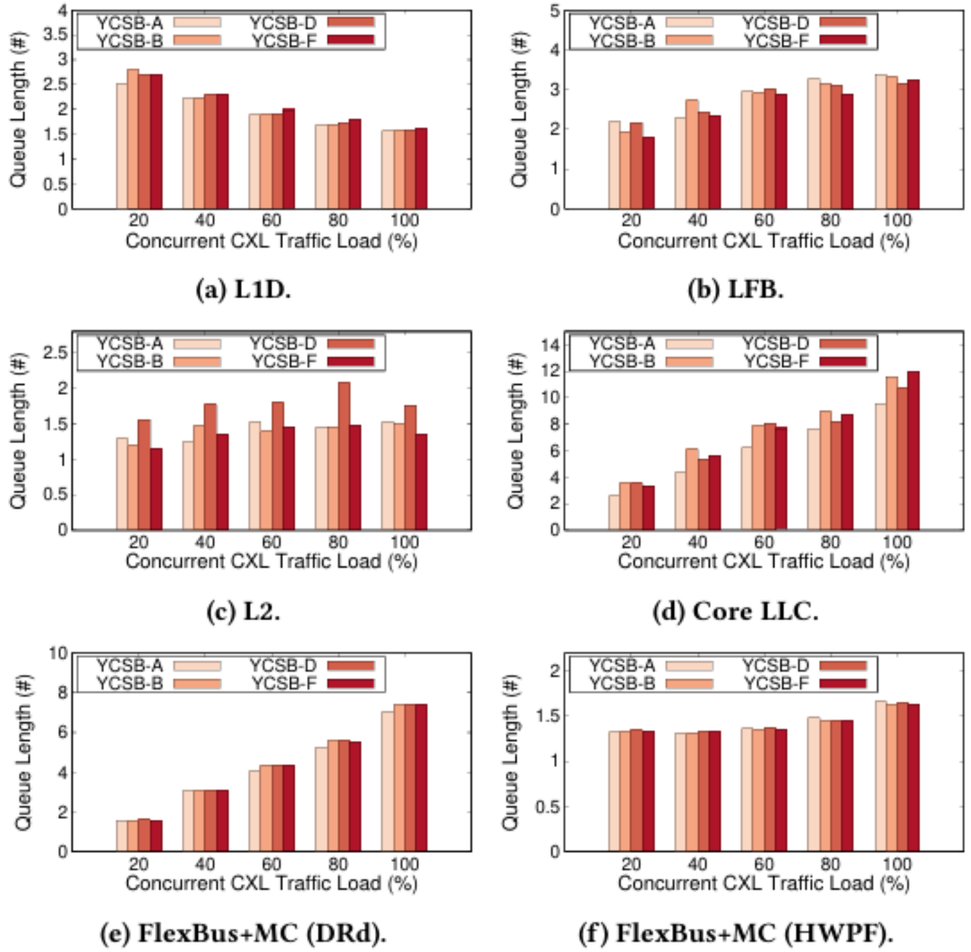


Figure 10: We present the queue length while increasing the CXL mFlow load from 20% to 100%. (a), (b), (c), and (d) present L1D, LFB, L2, and core-accessed LLC queue length. (e) and (f) present the FlexBus+MC queue length for DRd and HWPF, respectively.

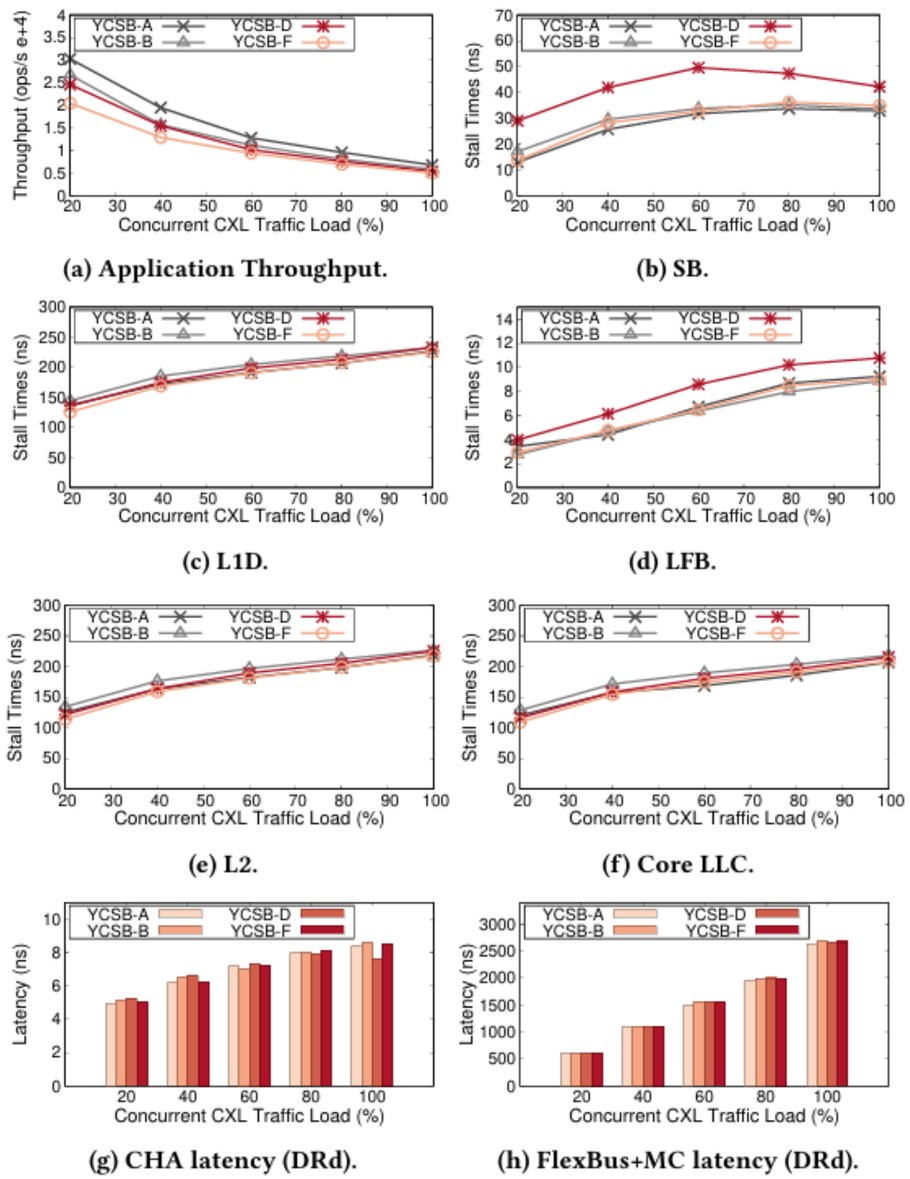


Figure 9: We present application performance and pipeline stall cycles while increasing the CXL mFlow load from 20% to 100%.

Bandwidth Partition & Data Locality

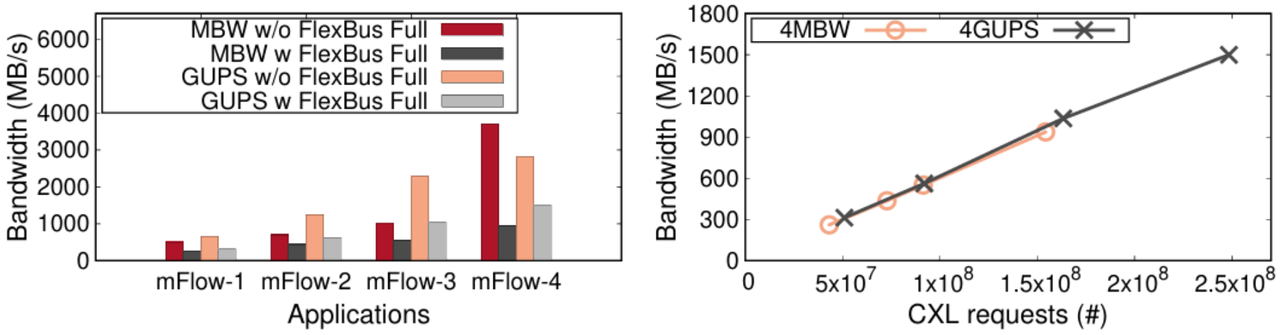
Case 5 - CXL Bandwidth Partition

- 4 concurrent MBW and 4 GUPS instances - FlexBus saturated.
- Bandwidth degradation is non-uniform: MBW-2 drops 37.7%, MBW-4 drops 74.7%
- Pearson correlation of 0.998 between CXL request frequency and actual bandwidth.
- When FlexBus is the bottleneck, PFBuilder's request frequency directly predicts each mFlow's bandwidth share.
- Gives operators a simple runtime formula for fairness estimation.

Case 6 - Data Locality Tracking

- PFMaterializer clusters hit distributions across snapshots to detect stable vs disturbed phases.
- Launching *554.roms_r* on CXL while *503.bwaves_r* runs locally: LLC misses worsen.
- Launching *519.lbm_r* instead (local): LLC misses decrease by 20.6% - friendlier neighbor.
- Mixed scenario (lbm + mcf + roms): compound interference visible as phase shifts in time-series.
- Enables operators to identify which co-located workload is the memory-locality culprit.

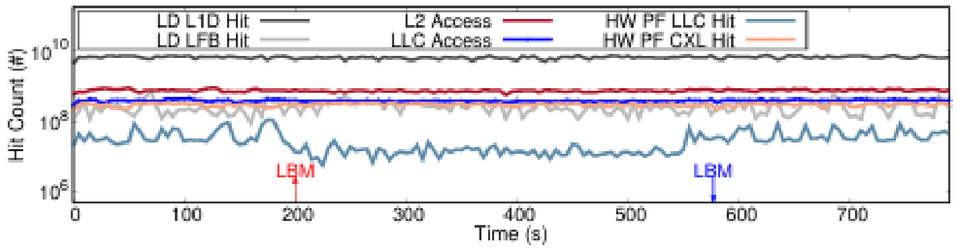
Case 5 - CXL Bandwidth Partition



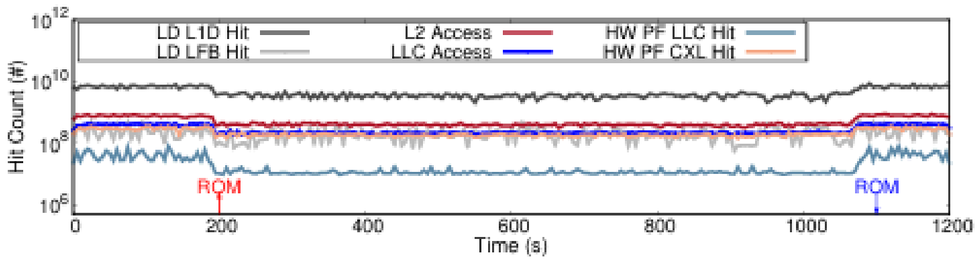
(a) mFlows bandwidth. (b) CXL BW and request frequency.

Figure 11: We examine the relationship between CXL bandwidth partition among concurrent CXL mFLows. (a) shows the results using 4 MBW programs with bandwidths of 500 MB/s, 700 MB/s, 1000 MB/s, and 3700 MB/s; using 4 GUPS programs with bandwidths of 650 MB/s, 1250 MB/s, 2200 MB/s, and 2800 MB/s. (b) shows the relationship between CXL memory request frequency and runtime bandwidth.

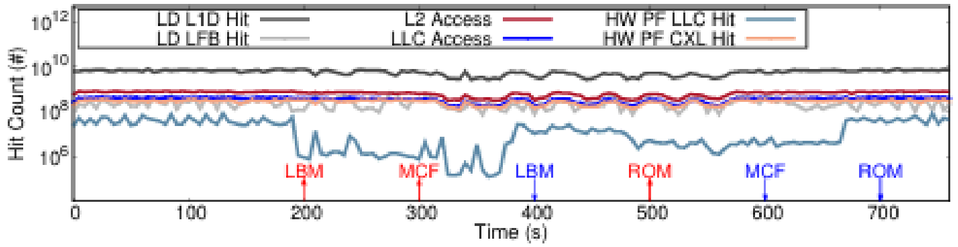
Case 6 - Data Locality Tracking



(a) 503.*bwaves_r* locality changes when launching 519.*lbm_r*.



(b) 503.*bwaves_r* locality changes when launching 554.*roms_r*.



(c) 503.*bwaves_r* data locality under multiple applications.

Figure 12: We monitor 503.*bwaves_r* application data locality changes when launching different applications. (a) Launch 519.*lbm_r* application accesses local memory. (b) Launch 554.*roms_r* application accesses CXL memory, further impacting 503.*bwaves_r* data locality. (c) A combination of 519.*lbm_r*, 505.*mcf_r*, and 554.*roms_r* applications accessing both local and CXL memory, introduce an additional interference effect to 503.*bwaves_r* data locality.

CASE STUDY 7

Performance Optimization Using PathFinder

PathFinder is used to understand and improve TPP (Transparent Page Placement) - a Linux kernel mechanism for CXL tiered memory.

YCSB-C **Query latency reduced 2.5%**

PFAnalyzer shows culprit path (DRd on LFB) queueing drops 3.6% with TPP

GUPS **Throughput improved 3.0×**

FlexBus+MC latency for DRd/RFO/HWPF drops 78–84% - bottleneck (LD on FlexBus) queue drops 96%

649.fotonik3d **Execution time reduced 14.3%**

DRd on L1D queue drops 82.1% - page migration shifts working set to local DRAM

Bonus: Dynamic TPP+Colloid - PathFinder guides page migration using per-phase DRd/RFO/HWPF miss ratios instead of fixed DRd latency → GUPS throughput improves another 1.1×

CRITICAL ANALYSIS

Strengths of PathFinder

S1

Elegant networking abstraction

Modeling the CPU + chipset as a Clos network is a powerful conceptual leap. It lets the authors borrow mature networking tools - traceroute, reverse traceroute, Little's Law, flow telemetry - and apply them to hardware profiling.

S2

Non-intrusive and lightweight

Uses existing hardware PMU counters - no firmware changes, no kernel patches beyond CHA PMU support. Adds only 1.3% CPU overhead and 38 MB memory, making it practical for production use.

S3

End-to-end path coverage

Uniquely traces from the CPU pipeline's store buffer all the way to the CXL DIMM - filling the gap between micro-arch profilers (on-chip only) and PCIe analyzers (fabric only).

S4

Actionable insights in real workloads

7 case studies with 77 applications covering SPEC, PARSEC, SPLASH, GAP, and Redis demonstrate clear optimization value - e.g., 3× GUPS improvement and 14.3% reduction in fotonik3d execution time.

S5

Open source and extensible

Built atop Linux Perf, open-sourced, and the InfluxDB/PFMaterializer layer allows new query workflows to be added without changing the profiling core.

CRITICAL ANALYSIS

Weaknesses & Limitations

W1

PMU counter gaps - RFO and DWr at L1D/LFB invisible

No PMU counters exist for RFO and DWr at L1D and LFB granularity. PFBuilder cannot distinguish demand vs prefetch RFOs at L2, and PFEstimator cannot break down stall cycles by access type below L2.

W2

Aggregated counters obscure per-flow isolation

When multiple mFlows share the same core, on-core PMU counters merge all their hits. PFBuilder can distinguish flows only beyond LLC (where DIMM-specific counters exist) - not inside the core pipeline.

W3

Intel-centric - limited portability

The design relies on Intel-specific structures like TOR in CHA and M2PCIe counters. AMD and ARM have functionally similar components, but the counter names and semantics differ. Porting requires significant re-mapping work.

W4

CXL Device PMU not fully exploited

Existing CXL DIMMs (Micron CZ120, Smart CMM-E3S, Samsung CMM-B/H) do not expose the 8 device-side performance diagnostic counters defined in CXL 3.0/3.1. The device-side half of the path remains a black box.

W5

Snapshot granularity tied to OS scheduler

Snapshots are taken at OS scheduling epoch boundaries - typically milliseconds. Sub-millisecond bursty CXL access patterns may be missed or averaged out, limiting diagnosis of transient bottlenecks.

Conclusion

- CXL memory is fundamentally different from local DRAM - it stalls pipelines, changes cache locality, and creates multi-stage congestion that no single existing tool could diagnose
- PathFinder's key insight: treat the server as a Clos network and apply networking telemetry - traceroute, reverse traceroute, queueing analysis, flow telemetry - to hardware
- Four techniques (PFBuilder, PFEstimator, PFAnalyzer, PFMaterializer) work together to give a complete, end-to-end view with only 1.3% CPU overhead
- Seven case studies confirm PathFinder can classify paths, isolate stall causes, detect interference, track locality, and guide optimization (up to 3× throughput improvement)
- Limitations exist in PMU counter gaps and Intel-specific design, but the framework is extensible as hardware evolves toward CXL 3.x

Thank You